



## Chapter 9

# Hashing\*

Up to this point, all the data structures we have seen in this course assume nothing but comparability about the input keys they are expected to manage. Hash functions, in general, are functions<sup>1</sup> from the key space into a lower dimensional space, which can be thought of as our memory storage.

*Example 1.* For example, assume the keys are taken from the integers, and the hash function  $h : \mathbb{N} \rightarrow [10]$  sends numbers to their residue modulo 10, namely  $h(x) = x \bmod 10$ . Now, one might use  $h$  and a 10-length array  $T$  to store numbers. Any time he would like to insert a number  $x$  into the structure, he would set  $T[h(x)] \leftarrow x$ .

However, this is not a “good” method, since two different keys with the same residue modulo 10 are mapped into the same cell in  $T$ , which we count as a collision:

**Definition 1** (collision.). Given a function  $h : U \rightarrow \star$ , we name any pair of different keys  $x \neq y \in U$  that are mapped by  $h$  to the same value, namely  $h(x) = h(y)$ , a collision.

Clearly, if we assume that no collisions are going to occur, i.e. all the given keys through the running of the program  $x_1, x_2, \dots, x_n$  satisfy that for any  $x_i \neq x_j$ , it follows that  $h(x_i) \neq h(x_j)$ , then the method in Example 1 provides a data structure that<sup>2</sup> supports access, insertion, and deletion in constant time. This gives us the intuition that the complexity of a general data structure which uses a hash function depends on the way it resolves collisions. In this recitation, we will present ways to handle collisions. The first is a heuristic called **open addressing** (or **closed hashing**). The second, **open hashing**, will also be analyzed, and we will show a characterization that, if satisfied, then we get a good running time in expectation. Finally, we will also show examples of function families that satisfy the characterization and examples of families that do not.

### 1 Open Addressing (Closed Hashing)

Open addressing uses a static  $m$ -length array  $T$ , hash function  $h$ , and handles collisions by probing cells in a specific order. For example, linear probing will first

<sup>1</sup>projections.

<sup>2</sup>Again, relative to assumptions on the input.

check if the cell at position  $h(x)$  is not occupied or already contains  $x$ . If it is, then  $h(x)$  is considered the correct place to store  $x$ . Otherwise, the next cell is probed, namely the cell at position  $h(x) + 1$ , and so on.

```

1 let  $i \leftarrow 0$ 
2 while  $T[h(x) + i] \neq \text{Null}$  do
3   |  $i \leftarrow i + 1$ 
4 end
5 return  $h(x) + i$ 

```

**Algorithm 1:** linear probing - access

## 2 Universal Hashing

Now that it is understood that collisions govern the running time, we are ready to study a classification of mappings which generate a low number of collisions in expectation.

**Definition 2.** Let  $\mathcal{H} = \{h_i : U \rightarrow [m]\}$  be a family of function from the domain  $U$  into  $[m] = \{0, 1, \dots, m-1\}$ .  $\mathcal{H}$  will be said universal if for any  $x \neq y \in U$ :

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{m}$$

**Question.** For  $x = y$  what is the probability that  $h(x) = h(y)$ ?

*Example 2.*  $\mathcal{H}$  is the set of all function from  $U \rightarrow [m]$ .

Picking a function randomly from the set of all functions is equivalent to picking the targets  $h(x)$ , for each  $x$ , independently. So the probability that  $h(x) = h(y)$  is as exactly as the probability, in the balls to bins experiment, that two specific balls ( $x$  and  $y$  in our case) will be thrown into the same bin. As we computed in the last recitation, that probability equals  $1/m$ .

*Example 3.*  $\mathcal{H}$  is all the inner products by a  $n$ -length binary vector. Namely,

$$\mathcal{H} = \{h_a : \mathbb{F}_2^n \rightarrow \mathbb{F}_2, h_a(x) = \langle a, x \rangle \mid a \in \mathbb{F}_2^n\}$$

Observe that in  $\mathbb{F}_2$ , the equivalence  $h(x) = h(y) \Leftrightarrow h(x) + h(y) = 0$  holds. If  $x \neq y$ , then there is at least one coordinate  $i$  such that  $x_i \neq y_i$ . In other words,  $x_i + y_i = 1$ . Thus:

$$\begin{aligned} h_a(x) + h_a(y) &= \langle a, x + y \rangle = a_i(x_i + y_i) + \sum_{j \neq i} a_j(x_j + y_j) \\ &= a_i + \sum_{j \neq i} a_j(x_j + y_j) \end{aligned}$$

Thus:

$$\begin{aligned}
 \Pr[h_a(x) = h_a(y)] &= \sum_{z \in \mathbb{F}_2} \Pr \left[ a_i = z \cap \left\{ \sum_{j \neq i} a_j(x_j + y_j) = z \right\} \right] \\
 &= \sum_{z \in \mathbb{F}_2} \Pr[a_i = z] \cdot \Pr \left[ \left\{ \sum_{j \neq i} a_j(x_j + y_j) = z \right\} \right] \\
 &= \sum_{z \in \mathbb{F}_2} \frac{1}{2} \cdot \Pr \left[ \left\{ \sum_{j \neq i} a_j(x_j + y_j) = z \right\} \right] \\
 &= \frac{1}{2} \sum_{z \in \mathbb{F}_2} \Pr \left[ \overbrace{\left\{ \sum_{j \neq i} a_j(x_j + y_j) = z \right\}}^1 \right] = \frac{1}{2}
 \end{aligned}$$

When in the second pass we used independence, and in the last, we used that probabilities sum to 1.

*Example 4.*  $\mathcal{H}$  is the set of all binary matrices :  $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$ .

Observe that the equivalence  $h(x) = h(y)$  implies an equivalence in any coordinate. Since the  $i$ th coordinate of  $h(x)$ , namely  $h(x)_i$ , is given by an inner product of the  $i$ th row of  $h$ , which is picked randomly from the  $n$ -length binary vectors, we get by Example 3 that the probability of equivalence in the  $i$ th coordinate is  $\frac{1}{2}$ , and since the rows are picked independently, we get:

$$\Pr[h(x) = h(y)] = \Pr \left[ \bigcap_i^k h(x)_i = h(y)_i \right] = \left( \frac{1}{2} \right)^k$$

**Exercise 1.**  $U$  is the set of all matrices  $\mathbb{F}_2^{n \times n}$  and  $h_X(A) = \text{Tr}(XA)$ .

*Solution 1.*

$$h_X(A) = \text{Tr}(XA) = \sum_i^n (XA)_{ii} = \sum_i^n X_{ij}A_{ji}$$

So, the hashing is equivalence to flattening  $A$  into an  $n \times n$  binary vector and then multiplying it by a randomly chosen vector. Which was proved in Example 3 to be a universal.

**Exercise 2.** Let  $\mathcal{H}$  be a universal family from  $U \rightarrow [m]$ . Show that  $\mathcal{H}^* : U \times U \rightarrow [m]$  is defined by:

$$h^*(x, y) = h(x) + h(y) \text{ for } h \in \mathcal{H}$$

is not universal.

*Solution 2.* Let  $x, y \in U$ . Consider the elements  $z_1 = (x, y)$  and  $z_2 = (y, x)$ . On one hand,  $z_1 \neq z_2 \in U \times U$ , on the other hand, for any  $h^* \in \mathcal{H}^*$  (defined with some  $h \in \mathcal{H}$ ) it holds that:

$$h^*(z_1) = h^*((x, y)) = h(x) + h(y) = h(y) + h(x) = h^*((y, x)) = h^*(z_2)$$

So, we get that with probability 1,  $h(z_1) = h(z_2)$ , and hence  $\mathcal{H}^*$  is not universal.

Equipped with universal hash functions, we can now say intelligent 'things' about the expected number of collisions. Specifically, denote by  $h \sim \mathcal{H}$  a sampled hash function. Let us denote by  $I$  the input:  $I = \{x_1, x_2, \dots, x_n\} \subset U$ . We can define for each  $x \in I$  the random variable  $C_x$ , which counts the number of collisions in  $I$  involving  $x$ . Namely:

$$C_x = |\{y \in I : h(x) = h(y)\}|$$

*Note, How to recall: at least 1 element for  $x$  itself, and  $\frac{n-1}{m}$  is what we would guess to have by throwing  $n-1$  balls to  $m$  bins.* **Claim 1.** If  $\mathcal{H}$  is universal hash family, then  $\mathbf{E}[C_x] = 1 + \frac{n-1}{m}$ .

*Proof.* For any  $x, y \in I$ , let  $X_{x,y}$  be the indicator of the event  $h(x) = h(y)$ . Observe that  $C_x = \sum_{y \in I} X_{x,y}$ . Therefore, by the linearity of expectation and  $\mathcal{H}$  being a universal hash family, we have:

$$\begin{aligned} \mathbf{E}[C_x] &= \mathbf{E}\left[\sum_{y \in I} X_{x,y}\right] = \sum_{y \in I} \mathbf{E}[X_{x,y}] = \sum_{y=x \in I} \mathbf{E}[X_{x,y}] + \sum_{y \neq x \in I} \mathbf{E}[X_{x,y}] \\ &= 1 + \sum_{y \neq x \in I} \Pr[h(x) = h(y)] \leq 1 + \frac{n-1}{m} \end{aligned}$$

□

## 2.1 Universal Hashing Table (Open Hashing)

Universal Hashing Table also defined by an  $m$ -array (table), and an hash function  $h$  which index keys  $x \in U$  to  $T[h(x)]$  cells, Yet in contrast to open addressing each cell might contains multiple keys by maintaining a linked list. Accessing to a stored key, or checking if it is in  $T$ , is done by iterating the linked list at suited cell. Appending a key, is done by first checking the key is not already stored in the table and then appending it to the tail of the suited linked list. ?? 2 exhibits the access routine, you are encouraged to complete the others.

**Question.** Fix a table, and consider  $x \in U$ . What will be the running time of each of the operation: Insert  $x$ , Access  $x$ , Delete  $x$ ?. (Linear at the number of keys in the list as  $T[h(x)]$ ).

**Question.** In hashing terms, what is the length of  $T[h(x)]$  equal to? The number of collision with  $x$ .

Hence, we infer that the expected running time is bounded by the expected number of collision with  $x$ . If  $h$  is sampled uniformly at random from a universal family  $\mathcal{H}$ , then by Claim 1, the expected running time is  $1 + \frac{n-1}{m} = O(1)$ .

## 2.2 How Hashing is Implemented in Python?

In most modern programming languages, the Dictionary data structure is implemented by a variation of Cuckoo hashing, which works as follows: We sample two functions from  $\mathcal{H}$ , denoted by  $h_1$  and  $h_2$ . Checking if an element  $x$  is in the

```

1 linked  $\leftarrow T[h(x)]$ 
2 for  $node \in linked$  do
3   | if  $node.value = x$  then
4   |   | return node
5   | end
6 end
7 return last node

```

**Algorithm 2:** Universal Hashing Table - access

```

1 if either  $T[h_1(x)]$  or  $T[h_2(x)]$  are empty then
2   | Pick an empty spot and set  $x$  there.
3   | return
4 end
5 else
6   |  $i \leftarrow$  flip a coin.
7   |  $z \leftarrow h_i(x)$ 
8   | set  $T[h_i(x)] \leftarrow x$ 
9   | Call Insert( $z$ ).
10 end

```

**Algorithm 3:** Cuckoo - Insert

table is done by checking if it is in either  $T[h_1(x)]$  or  $T[h_2(x)]$ . Inserting it into the table is done by the following:<sup>3</sup>

Roughly speaking, the algorithm fails when we close a cycle, namely after several recursive calls, back to the point where we were called to insert  $x$ . The idea of the analysis is that if the functions  $h_1, h_2$  were picked randomly, or very similar to randomly (i.e., from  $k$ -universal families), that event is unlikely, and the number of scanned spots is constant.

### 3 Perfect Hashing

In the past week, we have seen how to store keys in hash tables so that the number of mapped keys in a specific cell is  $O(1)$  in expectation. The table is constructed using a hashing function  $h : \text{key space} \rightarrow m\text{cells}$ , randomly chosen from a universal hash function family. This function maps keys to cells, and in each cell, the keys are stored using a linked list. The cost of supported subroutines depends on the length of the list. We named any pair of different keys  $x \neq y$  that are mapped to the same cell in the table, namely  $h(x) = h(y)$ , a collision.

Perfect hashing is a method to ensure that no collision occurs, it works only if all keys are given in advance and they are unique, meaning that the table doesn't support insertion. The idea is as follows, we sample an hash function, and then check if, for all  $x, y$  in the input, it holds that  $h(x) \neq h(y)$ . If so then we continue. Otherwise we repeat.

**Question.** What is the probability of choosing  $h$  with no collisions on the first trial? Notice that the answer depends on  $m$ . (To see this, imagine the case where

---

<sup>3</sup>Here we lie, a slightly clever (but not too mach) rule is used to pick  $i$ .

```

1 let collision  $\leftarrow$  True
2 while collision do
3   collision  $\leftarrow$  False
4   let  $T$  be array at length  $m$ 
5    $h \leftarrow$  sample uniformly random from universal hash family  $\mathcal{H}$ 
6   for  $x \in x_1, x_2, \dots, x_n$  do
7     if  $T_{h(x)}$  is not empty then
8       collision  $\leftarrow$  True
9       break the for-loop
10    end
11    else
12       $T_{h(x)} \leftarrow x$ 
13    end
14  end
15 end
16 return  $T, h$ 

```

**Algorithm 4:** perfect-hashing( $x_1, x_2, \dots, x_n$ )

$m = 1$ . In this case, there must be collisions.) Therefore, the correct question is: for what values of  $m$  do we succeed in finding a hash function with no collisions on the first trial? Let  $X_{x,y}$  be the indicator of the event  $h(x) = h(y)$ . The expected number of collisions is then:

$$\mathbf{E} \left[ \sum_{x \neq y} X_{x,y} \right] = \sum_{x \neq y} \mathbf{E} [X_{x,y}] = \binom{n}{2} \frac{1}{m}$$

Now, we would like to answer for what value of  $m$  there is no collision. Therefore, if we take  $m = n^2$ , then the expected number of collisions is less than  $1/2$ . By the Markov inequality, the probability of having more than one collision is less than:

$$P \left( \sum_{x \neq y} X_{x,y} > 1 \right) \leq \mathbf{E} \left[ \sum_{x \neq y} X_{x,y} \right] = \frac{1}{2}$$

And therefore the expected number of rounds is less than:

$$\mathbf{E} [\text{rounds}] = \sum_{t=0}^{\infty} t \Pr[t \text{ rounds}] \leq \sum_{t=0}^{\infty} t \frac{1}{2^{t-1}} = O(1)$$

**Question.** What is the space complexities? We have to allocate an array at length  $m$  which is  $\Theta(n^2)$  memory. Is that good? So remember that in standard hash tables, the expected number of elements that were hashed into the same cell as the key  $x$  is

$$1 + \frac{n-1}{m}$$

Taking  $m = \Theta(n)$  is enough to ensure that the expected running time of insertion/deletion/access is  $O(1)$ . This raises the question of whether the space complexity of perfect hashing can be reduced to linear.

### 3.1 Perfect Hashing in Linear Space

The idea is as follows: we will use a two-stage hashing process. In the first stage, keys will be mapped to hash tables instead of cells. Each hash table will be constructed using perfect hashing and may require a space that is quadratic in the number of elements stored in it (which were mapped to it in the first stage). Therefore, if we denote by  $n_i$  the number of elements mapped to the  $i$ th hash table, the space cost will be  $\sum_i n_i^2$ . Instead of starting over when a collision occurs, we will do so when  $\sum_i n_i^2 > 4n$ . So, now it's left to show that we expect  $\sum_i n_i^2$  to be

```

1 let toomanycollisions  $\leftarrow$  True
2 while toomanycollisions do
3   toomanycollisions  $\leftarrow$  False
4   let  $T$  be array at length  $m$ 
5   initialize any  $T_i$  to be an empty linked list.
6    $h \leftarrow$  sample uniformly random from universal hash family  $\mathcal{H}$ 
7   for  $x \in x_1, x_2, \dots, x_n$  do
8      $T_{h(x)}.\text{insert}(x)$ 
9      $T_{h(x)}.\text{size} = T_{h(x)}.\text{size} + 1$ 
10  end
11  if  $\sum_i T_{h(i)}.\text{size}^2 \geq \mu$  then
12    toomanycollisions  $\leftarrow$  True
13  end
14 end
15 let  $H$  be an array at length  $m$ 
16 for  $i \in [m]$  do
17    $T_i, h_i \leftarrow$  hash the elements in  $T_i$  using
18   perfect hashing.
19 end
20 return  $T, h$ 

```

**Algorithm 5:** perfect-hashing-linear-space( $x_1, x_2, \dots, x_n$ )

linear, which implies that the expected number of rounds is constant.

$$n_i^2 = 2 \binom{n_i}{2} + n_i$$

On the other hand,  $\sum_i \binom{n_i}{2}$  is exactly the number of collisions, as for any  $i$ ,  $\binom{n_i}{2}$  counts the number of distinct pairs in the  $i$ th table, which is equivalent to counting

the number of  $x \neq y$  such that  $h(x) = h(y) = i$ . Thus,

$$\begin{aligned} \mathbf{E} \left[ \sum_i n_i^2 \right] &= \mathbf{E} \left[ \sum_i 2 \binom{n_i}{2} + n_i \right] = 2 \cdot \mathbf{E} [\text{collisions}] + \mathbf{E} \left[ \overbrace{\sum_i n_i}^n \right] \\ &= 2 \cdot \binom{n}{2} \frac{1}{m} + n \end{aligned}$$

Therefore, by choosing  $m = 4n$  for the first stage, the probability of failing to choose a proper hash function is less than  $1/2$ .